

PROJECT DOCUMENTATION

DEVELOPMENT OF SMART EXAMINATION MONITORING PLATFORM WITH INTEGRITY ANALYSIS & REPORTING SYSTEM - EXAMGUARD

1. INTRODUCTION AND PROJECT BACKGROUND

1.1 Introduction

As part of my Infosys Virtual Internship, I was assigned a project based on online examination monitoring. The project required me to understand the challenges involved in monitoring students during online examinations and to develop a software-based solution that could record relevant examination activities and present them in an organized manner.

During the internship, I gained exposure to applying programming and data-related concepts to a practical software problem. The project provided an opportunity to work with technologies such as Python, Flask, SQLite, OpenCV, Data Science techniques, AI-based reporting, and Streamlit.

The assigned project was developed as ExamGuard – Smart Examination Monitoring Platform, with the official project title “Development of Smart Examination Monitoring platform with Integrity Analysis & Reporting System.”

1.2 Project Background

Online examinations require invigilators to monitor multiple students and identify activities that may require further review. Continuous manual monitoring can make it difficult to record every relevant event in a structured manner.

ExamGuard was developed as an integrated platform to support this process. The system monitors face presence through the webcam.

- Captures selected browser-related activities.
- Records examination events and session information.
- Calculates a rule-based integrity score.
- Analyses examination data using Data Science techniques.
- Groups examination sessions using K-Means clustering.
- Provides incident and evidence management.
- Generates AI-assisted examination reports.
- Presents analysis and reports through a Streamlit dashboard.

The platform follows four main concepts:

Monitor → Capture → Analyse → Report

- **Monitor** : Observe relevant examination activities.
- **Capture** : Record selected events and examination information.
- **Analyse** : Process the collected information and identify patterns.
- **Report** : Present the results through dashboards and AI-assisted reports.

ExamGuard is intended to assist invigilators during examination review. The recorded events, scores, and reports provide supporting information, while final interpretation and decisions remain with the responsible human reviewer.

1.3 Learning Background

The Infosys Virtual Internship provided practical exposure to applying software development concepts to an assigned real-world-oriented project. I strengthened my skills in Python, Flask, SQLite, OpenCV, Pandas, Scikit-learn, K-Means, LangChain, and Streamlit, along with system integration, testing, debugging, and documentation. The project helped connect my academic knowledge in Artificial Intelligence and Data Science with practical implementation.

2.PROBLEM IDENTIFICATION AND PROJECT IDEA

2.1 Problem Identification

Online examination monitoring can become difficult when an invigilator has to observe multiple students and different activities simultaneously.

The main problems identified were:

- Continuous manual monitoring is difficult.
- Important browser-related activities may be missed.
- Face absence events may be difficult to track consistently.
- Examination events may not be maintained in a structured manner.
- Reviewing multiple recorded events can be time-consuming.
- Raw events alone do not provide an organized analytical view.

These challenges created the need for an integrated monitoring and analysis platform.

2.2 Problem Statement

The problem addressed by ExamGuard is:

How can relevant online examination activities be recorded, organized, analyzed, and presented in a structured manner to support invigilator review?

The system addresses this by combining monitoring, event recording, integrity analysis, evidence management, and reporting.

2.3 Project Idea

The idea behind ExamGuard was to develop a single platform that combines examination monitoring and analysis.

The main functions are:

- Face Monitoring: Uses OpenCV and webcam input to monitor face presence.

- Browser Monitoring: Records selected browser-related events.
- Integrity Scoring: Applies predefined penalties to selected browser events.
- Data Science Analysis: Uses K-Means clustering to identify patterns in examination-session data.
- Incident & Evidence Management: Organizes incidents using Pending, Reviewed, and Dismissed statuses.
- AI Reporting: Generates natural-language examination reports using the AI reporting component.
- Dashboard: Uses Streamlit to present scores, analytics, and reports.

3.PROBLEM OBJECTIVES & SCOPE

3.1 Objectives

The main objectives I focused on while developing ExamGuard were:

- To develop a web-based platform for online examination monitoring.
- To monitor the presence of the student using the webcam.
- To record selected browser-related examination activities.
- To calculate an integrity score based on recorded rule-based events.
- To analyse examination data and identify patterns using K-Means clustering.
- To provide incident and evidence management for recorded events.
- To generate AI-assisted reports for examination sessions.
- To provide a dashboard for viewing monitoring and analytical information.
- To support invigilators in reviewing examination sessions more systematically.

3.2 Project Scope

The scope of ExamGuard includes:

- Student registration and login.
- Examination session management.
- Face presence monitoring using OpenCV.
- Browser activity event logging.
- Rule-based integrity score calculation.
- Incident status management.
- Evidence management.

- Data analysis using machine learning techniques.
- K-Means-based student/session pattern grouping.
- AI-assisted integrity report generation.
- Dashboard-based visualization and review.
- Export of relevant examination data in JSON/CSV formats.

The system is intended to assist human invigilators by organizing examination information. The final interpretation of recorded events remains with the responsible human reviewer.

4.EXISTING SYSTEM & PROPOSED SOLUTION

4.1 Existing System

Before developing ExamGuard, I considered the difficulties associated with conventional online examination monitoring approaches.

The existing approach may involve:

- Manual observation of students during an online examination.
- Separate tools for examination, communication, and monitoring.
- Manual identification of suspicious examination events.
- Limited structured records for later review.
- Difficulty in reviewing multiple types of events together.
- Dependence on continuous attention from the invigilator.
- Lack of integrated analytical information about an examination session.

4.2 Proposed Solution

To address these issues, I developed **ExamGuard**, an integrated online examination monitoring platform that provides:

- **Face Presence Monitoring** – Webcam-based face detection and presence recording.
- **Browser Activity Monitoring** – Records tab switching, copy/paste, right-click, F12, Developer Tools, and View Source events.
- **Integrity Score** – Applies predefined penalties to selected browser events.
- **Incident & Evidence Management** – Organizes incidents using Pending, Reviewed, and Dismissed statuses with supporting evidence.
- **Data Science Analysis** – Uses feature scaling and K-Means clustering to identify examination-session patterns.

- **AI-Based Reporting** – Uses LangChain to generate natural-language examination summaries.
- **Dashboard & Export** – Streamlit dashboard for analysis, review, and JSON/CSV data export.

4.3 Existing System vs Proposed System

Existing Approach	Proposed ExamGuard
Manual report preparation	AI-assisted report generation
Events may be reviewed separately	Events are organized in one platform
Limited structured event records	Examination events are recorded
Manual interpretation of raw events	Systematically provides a summarized
Limited pattern analysis	K-Means-based analysis

5. TECHNOLOGY STACK

5.1 Programming Language

Python

Python was used as the primary programming language because it provides libraries and frameworks suitable for web development, computer vision, data processing, machine learning, and AI-based applications.

Python was used for:

- Application development.
- Backend processing.
- Database interaction.
- Face monitoring.
- Data analysis.
- Integrity score calculation.
- Machine learning implementation.
- AI reporting.
- Data export and processing.

Python Version: Python 3.13.5

The exact Python version used for the final project environment should be mentioned here based on the version installed in the development environment.

5.2 Web Application Development

- **Flask** was used as the main web framework for user authentication, examination sessions, application routes, monitoring operations, browser-event logging, scoring, and submission.
- **HTML, CSS and JavaScript** were used for the examination interface, user interaction, browser-event detection, and camera-related functionality.

5.3 Database

- **SQLite** was used to store user accounts, examination sessions, events, browser activities, integrity scores, and incident information. It was suitable for the project's development and testing environment.

5.4 Computer Vision

- **OpenCV** was used for webcam-based face presence monitoring. A **Haar Cascade classifier** was used for face detection and recording relevant monitoring information.

5.5 Data Processing and Machine Learning

Pandas was used for data processing and preparation.

- **Scikit-learn** was used for StandardScaler and K-Means clustering. K-Means was applied to group examination sessions based on selected features and identify patterns.

5.6 AI-Based Reporting

- **LangChain** was used in the AI reporting component to process examination information and generate natural-language reports for invigilator review.

5.7 Dashboard and Visualization

- **Streamlit** was used to develop the analytical dashboard for displaying examination events, integrity scores, data analysis, visualizations, AI-generated reports, and export information.

6.SYSTEM ARCHITECTURE

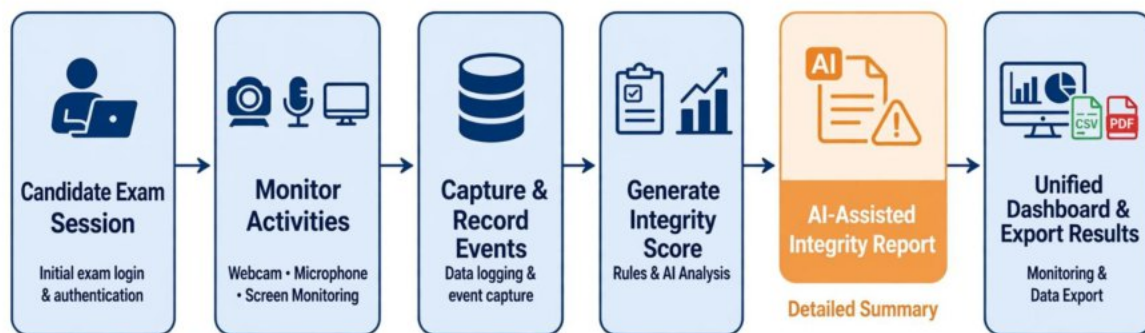
6.1 System Architecture

The ExamGuard architecture connects the examination, monitoring, database, analysis, and reporting components into a single workflow.

- **Flask Application:** Acts as the main application layer and manages examination-related operations.
- **Monitoring Layer:** Handles webcam-based face presence monitoring and selected browser activity monitoring.
- **SQLite Database:** Stores user, examination session, event, score, and incident-related information.
- **Integrity Analysis:** Processes recorded events and calculates the rule-based integrity score.

- **Incident & Evidence Management:** Organizes recorded incidents and their supporting evidence for review.
- **Data Science Layer:** Processes examination data and applies K-Means clustering to identify patterns.
- **AI Reporting:** Uses the LangChain-based component to generate natural-language examination reports.
- **Streamlit Dashboard:** Presents scores, analytics, patterns, and reports for invigilator review.

Figure 6.1: ExamGuard System Architecture



7. SYSTEM MODULES & SERVICES

7.1 User and Examination Management

Provides the basic functionality for conducting an online examination.

- User registration and login.
- Examination session management.
- Start, pause, resume, and submit examination.
- Saving examination answers.
- Maintaining session information.

7.2 Face Presence Monitoring

Implemented using OpenCV for webcam-based monitoring.

- Processes webcam feed.
- Detects face presence.
- Records periods when the face is not detected.
- Stores face-monitoring information for review.

7.3 Browser Activity Monitoring

Records selected browser-related activities during the examination.

- Tab switching
- Right-click
- Copy/Paste
- F12
- Developer Tools
- View Source

7.4 Integrity Score Module

Calculates a rule-based integrity score using predefined penalties for selected browser events.

Browser Event	Penalty
Tab Switching	5
Right Click	2
Copy/Paste	3
F12	5
Developer Tools	10
View Source	5

The score is used as an analytical indicator for examination review.

7.5 Incident and Evidence Management

Organizes events that require further review.

- Records incidents and associated evidence.
- Tracks review status.
- Supports Pending, Reviewed, and Dismissed statuses.

7.6 Data Science Analytics

Processes examination data and applies machine learning techniques.

- Data preparation and feature scaling.
- K-Means clustering using selected examination features.
- Session grouping and pattern analysis.
- Analytical charts and visualizations.

7.7 AI-Based Reporting

Uses LangChain to generate natural-language summaries from examination-related information.

- Processes recorded examination information.
- Generates AI-assisted reports.
- Supports invigilator review.

7.8 Dashboard

The Streamlit dashboard provides a centralized view of:

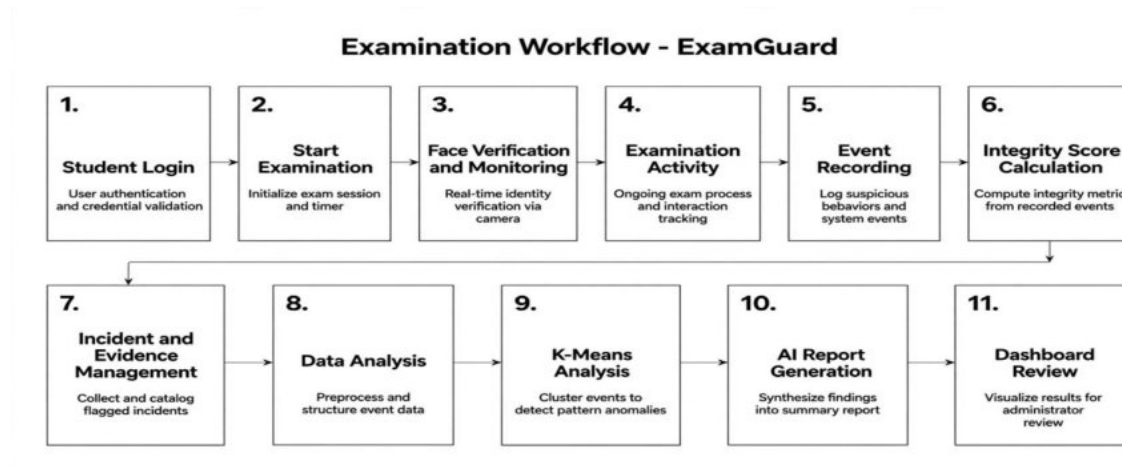
- Integrity scores and examination events.
- Analytical patterns and cluster information.
- AI-generated reports.
- Export-related information.

8.SYSTEM WORKFLOW & IMPLEMENTATION

8.1 Examination Workflow

- 1.Student Login – Student logs into the examination platform.
- 2.Start Examination – An examination session is created and tracked.
- 3.Face Monitoring – Webcam-based face presence monitoring begins.
- 4.Examination Activity – Student answers questions while selected browser activities are monitored.
- 5.Event Recording – Face-monitoring information and browser events are recorded.
- 6.Integrity Score – Predefined penalties are applied to relevant browser events.
- 7.Incident & Evidence Management – Relevant incidents and supporting evidence are organized for review.
- 8.Data Analysis – Examination data is processed and selected features are standardized.
- 9.K-Means Analysis – Examination sessions are grouped based on selected features and observed patterns.
- 10.AI Report Generation – Examination information is processed to generate a natural-language report.
- 11.Dashboard Review – The Streamlit dashboard presents analytical information and reports for invigilator review.

Figure 8.1: Examination Workflow



9. DATA SCIENCE & AI COMPONENTS

9.1 Data Processing

ExamGuard processes examination-session data such as:

- Session information.
- Browser activity counts.
- Face-monitoring information.
- Integrity scores.
- Incident information.

9.2 Feature Preparation and Scaling

Selected features used for analysis include:

- Integrity score.
- Total events.
- Browser activity counts.
- Face-related counts.
- Face presence ratio.

StandardScaler is applied to standardize the numerical features before clustering.

9.3 K-Means Clustering

K-Means clustering is used to identify patterns in examination sessions.

- Clusters: 3
- Preprocessing: StandardScaler
- Input: Selected examination-session features

- Analytical categories: High, Medium, Low

The categories are used for analysis and do not represent automatic disciplinary decisions.

9.4 AI-Based Reporting

A LangChain-based workflow generates readable examination reports by:

- Summarizing relevant examination information.
- Presenting recorded activities clearly.
- Supporting invigilator review.

10. CHALLENGES FACED

10.1 Technology Integration

Integrating Flask, SQLite, OpenCV, Pandas, Scikit-learn, LangChain, and Streamlit into one application required component-wise testing and integration.

10.2 Face and Browser Monitoring

Handling webcam input, face absence, and browser events required careful testing. I implemented these components separately and connected them to the examination workflow.

10.3 Integrity Score

Converting recorded events into a consistent score was addressed by defining fixed penalties and applying them to recorded events.

10.4 Database and Session Management

Maintaining the correct relationship between users, examination sessions, and recorded events required proper session handling and database testing.

10.5 Data Science and AI Integration

K-Means required feature preparation and scaling before clustering. AI reporting required connecting the correct examination-session data to the reporting workflow.

11. LIMITATIONS & BETTER SOLUTIONS

11.1 Limitations

- Face detection may be affected by poor lighting, camera quality, or face visibility.
- Browser monitoring covers only selected suspicious activities.
- Integrity scores depend on predefined penalty rules.
- K-Means results depend on the selected features and available data.

- AI-generated reports require human review before making decisions.
- The current system has been tested on a limited scale.

11.2 Better Solutions

- Use more robust face and behaviour detection techniques.
- Add better handling for camera and network interruptions.
- Expand browser and examination activity monitoring.
- Improve scoring using contextual and anomaly-based analysis.
- Train and evaluate models using larger and more diverse datasets.
- Strengthen authentication, data security, and privacy controls.
- Support cloud deployment for larger-scale examinations.

12. RESULTS & CONCLUSION

12.1 Project Outcome

The development of ExamGuard resulted in a functional online examination monitoring platform that brings different monitoring and analysis activities together in one system. The completed system supports:

- Student registration, login, and examination session management.
- Webcam-based face presence monitoring.
- Selected browser activity recording.
- Rule-based integrity score calculation.
- Incident and evidence management.
- K-Means-based examination-session analysis.
- AI-assisted report generation.
- Streamlit-based dashboard for reviewing examination information.
- JSON/CSV export of relevant examination data.

The implemented features provide a structured way to record and review examination-session information.

12.2 What I Learned

Working on this project helped me gain practical experience beyond individual programming concepts. I learned how different technologies can be connected to build a complete application and how testing and debugging are important when multiple components interact.

Through the project, I gained experience in:

- Python, Flask, and SQLite application development.
- OpenCV-based computer vision.
- Data processing using Pandas and Scikit-learn.
- K-Means clustering and feature preparation.
- AI-assisted reporting using LangChain.
- Dashboard development using Streamlit.
- Database, session, and component integration.
- Testing, debugging, documentation, and understanding system limitations.

12.3 Conclusion

ExamGuard was developed as an online examination monitoring platform to help invigilators monitor and review important examination activities in a more organized manner. The project brought together Python, Flask, SQLite, OpenCV, Data Science techniques, K-Means clustering, LangChain, and Streamlit to build different parts of the system.

Through this project, I gained practical experience in integrating multiple technologies into a single application, handling examination data, identifying recorded activities, analysing session information, and presenting the results through a dashboard. The system provides integrity scores, incident and evidence information, analytical results, and AI-assisted reports to support the review process.

Overall, the project helped me understand how the concepts I learned in Artificial Intelligence and Data Science can be applied to a practical software system. ExamGuard is designed to assist invigilators by organizing relevant examination information, while the final interpretation and decisions remain with the human reviewer.